Search...
Tutorials
Practice
Jobs
Sign In

Designing UPI - System Design

Last Updated : 28 Mar, 2026

Designing a Unified Payments Interface (UPI) system involves creating an architecture that enables real-time inter-bank transactions through a seamless and secure platform. Key components include user interfaces, a central UPI switch managed by the National Payments Corporation of India (NPCI), backend systems of participating banks, and third-party service providers.

- The system supports core functionalities such as user registration and authentication, payment initiation and authorization, inter-bank transaction processing, and real-time settlement.
- Security measures like encryption and multi-factor authentication ensure compliance and protect against fraud, making UPI a reliable and efficient payment solution.
- The UPI is a real-time payment system that facilitates inter-bank transactions by instantly transferring funds between two bank accounts on a mobile platform.

1. System requirements

This section outlines the functional and non-functional requirements for designing a UPI system.

1. Functional Requirements

Below are the functional requirements for UPI system design:

- **User Registration and Authentication:** Users should be able to register and create a UPI account. Provide secure authentication mechanisms (e.g., PIN, biometric authentication).
- **Bank Account Linking:** Users should be able to link multiple bank accounts to their UPI profile. Facilitate the management of linked bank accounts (e.g., add, remove, and update account details).
- **Payment Address Management:** Users should be able to create and manage Virtual Payment Addresses (VPAs).
- **Money Transfer:** Support person-to-person (P2P) transfers. Support person-to-merchant (P2M) payments. Enable scheduled and recurring payments.
- **Transaction History:** Provide users with a detailed transaction history, including status and timestamps.
- **Notifications:** Send real-time notifications for all transactions (e.g., payment success, failure).
- **Payment Requests:** Allow users to request payments from other UPI users.
- **QR Code Payments:** Generate and scan QR codes for quick and easy payments.
- **Bill Payments:** Support utility bill payments through the UPI interface.

2. Non-Functional Requirements

Below are the non-functional requirements for UPI system design:

- **Scalability:** The system should handle a large number of users and high transaction volumes. Support horizontal scaling to manage increasing load efficiently.
- **Performance:** Ensure low latency for transaction processing and user interactions. Provide real-time transaction processing capabilities.
- **Reliability:** Ensure high availability with minimal downtime. Implement failover mechanisms to handle system failures gracefully.
- **Security:** Implement robust security measures to protect user data and financial transactions. Ensure compliance with financial regulations and standards (e.g., PCI-DSS).
- **Consistency:** Ensure consistency in transactions to avoid discrepancies and double-spending. Implement mechanisms to ensure eventual consistency in a distributed system.
- **Compliance:** Adhere to regulatory requirements and guidelines set by financial authorities. Ensure data privacy and protection in compliance with relevant laws.
- **Usability:** Design a user-friendly interface that is easy to navigate. Ensure accessibility for users with different levels of technical expertise.

2. Capacity Estimation

Below is the capacity estimation for UPI system design:

1. Traffic Estimate

This section calculates the expected number of users and transactions.

- **Users:** 100 million users
- **Daily Active Users (DAU):** 10 million users
- **Transactions per User per Day:** 5 transactions
- **Total Transactions per Day:** 50 million transactions

2. Storage Estimate

This section estimates how much data will be stored over time.

- **Average Transaction Size:** 500 bytes
- **Daily Data:** 50M × 500 bytes = ~25 GB
- **Monthly Data:** 25 GB × 30 = ~750 GB
- **Yearly Data:** 750 GB × 12 = ~9 TB

3. Bandwidth Estimate

This section calculates the amount of data transferred daily.

- **Request Size:** 1 KB
- **Response Size:** 1 KB
- **Total Requests per Day:** 50 million
- **Total Data per Day:** ~95 GB

4. Memory Estimate

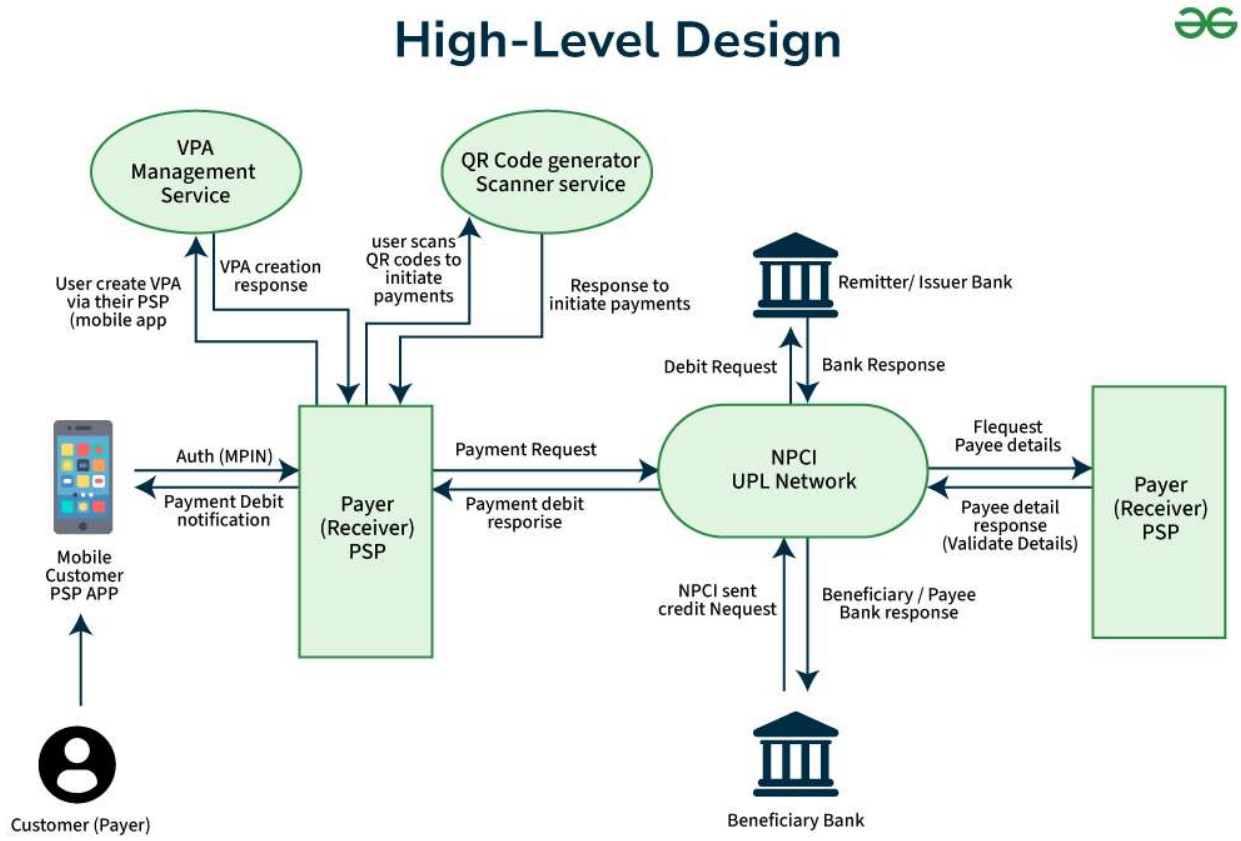
This section estimates caching requirements for active users.

- **Active Users Cached:** 1 million
- **Data per User:** 1 KB
- **Total Memory Required:** ~1 GB

High-Level Design(HLD)

A high-level diagram for a UPI system provides an overview of the main components and their interactions. At the top layer, users interact with UPI through mobile applications or web interfaces provided by banks or third-party service providers (e.g., Google Pay, PhonePe). These interfaces connect to the UPI service, a centralized system managed by the National Payments Corporation of India (NPCI).

- The UPI service acts as a switch that handles transaction requests, routing them to the appropriate bank's backend systems. Each participating bank has its own infrastructure to manage account verification, balance checks, and transaction processing.
- The UPI switch facilitates real-time communication and settlement between the payer’s and payee’s banks. Additionally, the system incorporates security mechanisms, including encryption and authentication, to ensure secure transactions.
- APIs allow third-party apps to integrate UPI functionalities, extending the system’s usability and accessibility. This high-level structure ensures efficient, secure, and seamless inter-bank transactions.



The diagram illustrates the flow of a Unified Payments Interface (UPI) transaction, which is a real-time payment system developed by the National Payments Corporation of India (NPCI). It shows the interaction between various entities involved in a UPI transaction. Here's a detailed explanation:

1. Entities

This section defines the key entities involved in a UPI transaction system.

- **Customer (Payer):** The individual initiating the payment.
- **Payer (Receiver) PSP:** The Payment Service Provider (PSP) of the payer, which facilitates the transaction.
- **Remitter/Issuer Bank:** The bank of the payer.
- **Beneficiary Bank:** The bank of the payee.
- **Payee (Receiver) PSP:** The Payment Service Provider of the payee.
- **NPCI UPI Network:** The central network that processes UPI transactions.
- **VPA Management Service:** Service for managing Virtual Payment Addresses (VPAs).
- **QR Code Generator/Scanner Service:** Service for generating and scanning QR codes for payments.

2. Flow of the Transaction

This section describes the step-by-step flow of a UPI transaction from initiation to completion.

VPA Creation

This step involves creating a unique Virtual Payment Address for the user.

- Customer creates a VPA using the PSP mobile app via the VPA Management Service.
- The request is processed and a response is returned confirming VPA creation.

Initiating Payment

This step starts the payment process using QR or other methods.

- Customer scans a QR code using the QR Code Generator/Scanner Service.
- The system processes the scan and initiates the payment request.

Payment Authorization

This step ensures secure user authentication before processing payment.

- Customer authorizes the payment using MPIN in the PSP app.
- Payer PSP sends the payment request to the NPCI UPI Network.

Processing Payment

This step handles debit and credit between banks.

- NPCI sends a debit request to the Remitter/Issuer Bank.
- Remitter Bank processes and responds to NPCI.
- NPCI sends a credit request to the Beneficiary Bank.
- Beneficiary Bank processes and responds to NPCI.

Payee Details Validation

This step validates the receiver’s details before completing the transaction.

- NPCI requests payee details from the Payee PSP.
- Payee PSP validates and sends a response back.

Completion of Transaction

This step finalizes the transaction and notifies the user.

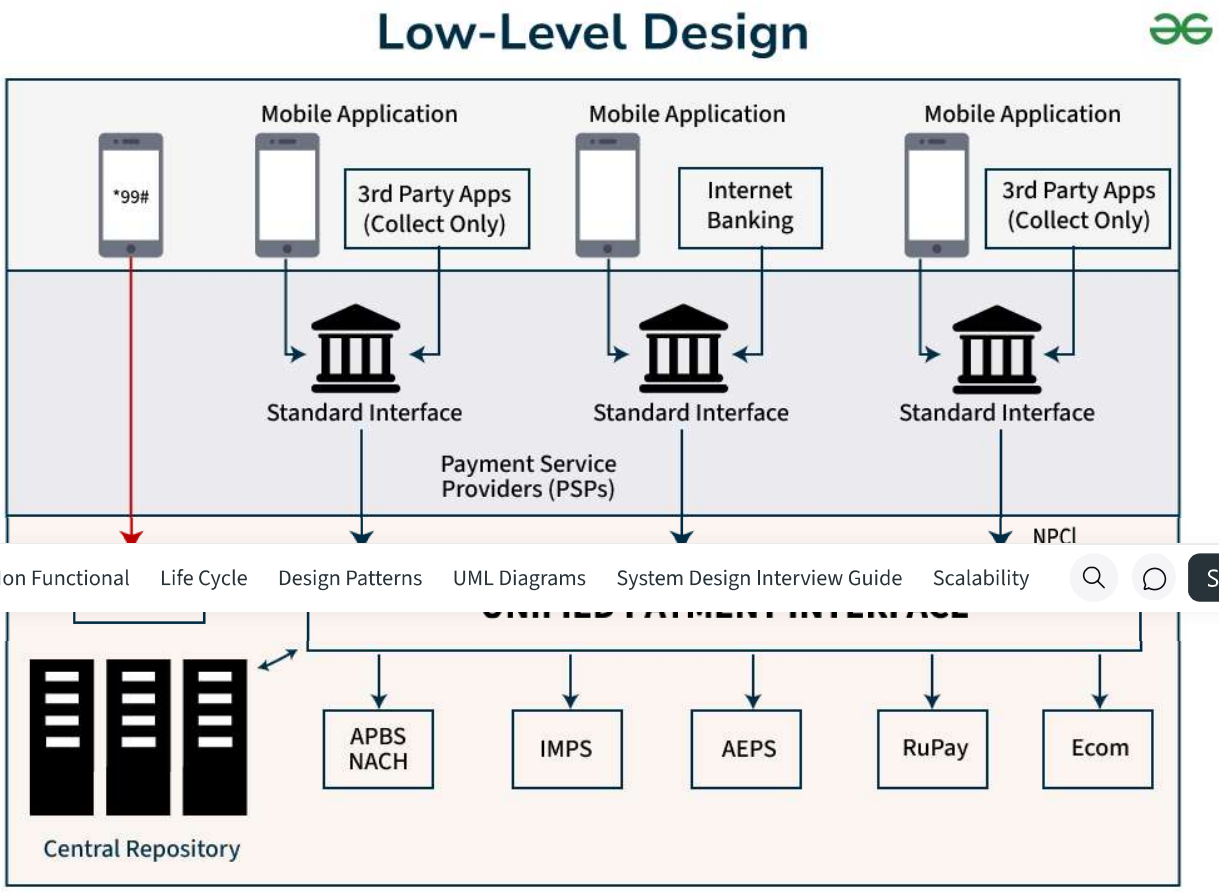
- NPCI sends the payment response to the Payer PSP.
- Payer PSP notifies the customer about successful or failed payment.

In Conclusion, The customer initiates a payment using their mobile app by either creating a VPA or scanning a QR code. The payer PSP sends a payment request to the NPCI UPI Network. The NPCI UPI Network processes the payment by interacting with the payer's and payee's banks. The transaction is completed, and the customer is notified of the payment status.

Low-Level Diagram(LLD)

A low-level diagram for a UPI system illustrates the detailed interaction between its core components. At the user level, we have mobile apps and web interfaces that allow users to register, authenticate, and initiate transactions. These interfaces communicate with the UPI service through APIs, which are crucial for handling requests and responses.

- The UPI service, managed by the National Payments Corporation of India (NPCI), acts as a central switch that routes transaction requests to the appropriate bank systems. Each participating bank has backend systems that verify user account details, check balances, and process transactions.
- The UPI switch ensures real-time transaction processing and settlement by securely relaying information between the payer's and payee's banks. Additionally, the system includes security modules for encryption and fraud detection, ensuring the integrity and confidentiality of transactions.
- Third-party service providers integrate with the UPI service via APIs to offer UPI functionalities within their apps, expanding the system's reach and usability.



ctional and Non Functional Life Cycle Design Patterns UML Diagrams System Design Interview Guide Scalability Sign In

The diagram illustrates the architecture and flow of the Unified Payments Interface (UPI) system, which is a real-time payment system developed by the National Payments Corporation of India (NPCI). It shows how different entities interact within the UPI ecosystem to facilitate transactions. Here's a detailed explanation:

1. Entities and Components

This section describes the key components and participants involved in the UPI ecosystem.

- **Mobile Application:** Interface used by users to perform transactions.
- **USSD (99#):* Enables banking transactions on basic phones without internet access.
- **3rd Party Apps (Collect Only):** Apps that can initiate payment collection requests.
- **Internet Banking:** Allows users to access banking services through web/mobile platforms.
- **Banks (Standard Interface):** Provide interfaces to interact with the UPI network.

- **Payment Service Providers (PSPs):** Facilitate transactions for users and merchants via UPI.
- **Unified Payment Interface (UPI):** Central platform that processes all UPI transactions.
- **Central Repository:** Stores transaction data and system-related information.
- **NPCI:** Governing body that manages and operates the UPI system.

2. Other Payment Systems

- **APBS/NACH:** Aadhaar Payment Bridge System/National Automated Clearing House.
- **IMPS:** Immediate Payment Service.
- **AEPS:** Aadhaar Enabled Payment System.
- **RuPay:** A domestic card payment network.
- **Ecom:** E-commerce transactions.

3. Flow of Transactions

This section describes how different components interact to process UPI transactions.

USSD (99#): This flow enables transactions without internet using basic mobile phones. Users initiate transactions using the *99# USSD service, and the request is sent to the Central Repository, which then forwards it to the UPI system for processing.

Mobile Applications: This flow handles transactions via smartphones and apps. Users initiate transactions through mobile applications such as third-party apps or internet banking apps, which interact with bank interfaces to process the payments.

Banks: This flow explains the role of banks in transaction processing. Banks provide standard interfaces to connect with the UPI system and act as intermediaries between users and the UPI network to facilitate transactions.

Payment Service Providers (PSPs): This flow describes how PSPs facilitate transactions. PSPs interact with both banks and the UPI system to ensure smooth, secure, and efficient processing of user transactions.

Unified Payment Interface (UPI): This flow explains the central processing system. The UPI platform coordinates with banks and PSPs to process transactions and ensures real-time settlement and execution.

Central Repository: This flow explains data storage and management. It stores transaction data and other relevant information and provides necessary data to the UPI system for processing.

Other Payment Systems: This flow shows integration with external systems. UPI interacts with systems like IMPS, AEPS, RuPay, and others to support various types of financial transactions.

The diagram shows how users can initiate transactions using different methods (USSD, mobile apps, internet banking). Banks provide a standard interface to interact with the UPI system. Payment Service Providers (PSPs) facilitate the transactions. The UPI system processes the transactions in real-time, interacting with banks and other payment systems. The Central Repository stores transaction data and interacts with the UPI system.

Microservices Used

Below are some microservices used in UPI System Design:

User Service

This service handles user registration and authentication.

- POST /register – Register a new user
- POST /login – Authenticate a user

Bank Service

This service manages bank account operations.

- POST /link-account – Link a bank account
- GET /balance – Fetch account balance

Transaction Service

This service handles money transfers and transaction records.

- POST /transfer – Transfer money
- GET /history – Retrieve transaction history

Notification Service

This service is responsible for sending updates to users.

- POST /notify – Send notifications to users

Scalability & Performance Optimization

This section describes techniques to ensure the system can handle increasing users and transaction load efficiently.

Horizontal Scaling

This approach increases capacity by adding more servers instead of upgrading a single machine.

- Microservices Architecture: Each service scales independently based on demand.
- Load Balancing: Distributes incoming requests across multiple instances.

Caching

This improves performance by storing frequently accessed data in memory.

- Redis: Caches user sessions, frequently accessed data, and transaction states.

Database Sharding

This distributes data across multiple databases to reduce load.

- Shard by User ID: Splits user data across different database instances.

Asynchronous Processing

This handles tasks in the background to improve responsiveness.

- Message Queues: Use Kafka or RabbitMQ for notifications and transaction processing.

Fault Tolerance

This ensures system reliability during failures.

- Circuit Breaker Pattern: Prevents cascading failures.
- Retry Mechanism: Handles temporary failures by retrying operations.

Comment

A aditipa... 17

Article Tags: System Design

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Careers
Contact Us
Corporate Solution
Campus Training

Explore
POTD
Practice Problems
Connect
Blogs
Upskill Courses

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
GATE
School Subjects
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Data Science
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Offline Centers
Noida
Bengaluru
Hyderabad
Kolkata

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved